

ASP.net Core problems:

Problem1:

Employee.cs (Model)

using System.ComponentModel.DataAnnotations;

```
namespace CollegePortalMvcApp.Models
{
    public class Employee
    {
        [Required(ErrorMessage = "Id is required")]
        public required int Id { get; set; }

        [Required(ErrorMessage = "Enter the required field")]
        public required string Name { get; set; }

        [Range(10000, 500000, ErrorMessage = "salary should be in between 10k to 500k")]
        public decimal Salary { get; set; }

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Invalid email address")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Joining date is required")]
        [DataType(DataType.Date)]
        public DateTime JoiningDate { get; set; }
    }
}
```

EmployeeController.cs (Controller)

using Microsoft.AspNetCore.Mvc;

using CollegePortalMvcApp.Models;

using Microsoft.AspNetCore.Mvc.ModelBinding;

```
namespace CollegePortalMvcApp.Controllers
{
    public class EmployeeController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Employee employee)
        {
            if (employee.JoiningDate > DateTime.Today)
            {
                ModelState.AddModelError("JoiningDate", "Joiningdate cannot be further");
            }

            if (ModelState.IsValid)
            {
                return View("Success", employee);
            }
            return View(employee);
        }
    }
}
```

```
}
```

```
Views(Create.cshtml,Success.cshtml)
```

```
Create.cshtml
```

```
@model CollegePortalMvcApp.Models.Employee
```

```
<h2>Employee Form Details</h2>
```

```
<form asp-action="Create" method="Post">
```

```
    <div>
```

```
        <label>Id</label>
```

```
        <input asp-for="Id"/>
```

```
        <span asp-validation-for="Id"></span>
```

```
    </div>
```

```
    <div>
```

```
        <label>Name</label>
```

```
        <input asp-for="Name"/>
```

```
        <span asp-validation-for="Name"></span>
```

```
    </div>
```

```
    <div>
```

```
        <label>Salary</label>
```

```
        <input asp-for="Salary"/>
```

```
        <span asp-validation-for="Salary"></span>
```

```
    </div>
```

```
    <div>
```

```
        <label>Email</label>
```

```
        <input asp-for="Email"/>
```

```
        <span asp-validation-for="Email" type="date"></span>
```

```
    </div><div>
```

```
        <label>JoiningDate</label>
```

```
        <input asp-for="JoiningDate"/>
```

```
        <span asp-validation-for="JoiningDate"></span>
```

```
    </div>
```

```
    <button type="submit">Submit</button>
```

```
</form>
```

```
,Success.cshtml)
```

```
@model CollegePortalMvcApp.Models.Employee;
```

```
<h2>Employee Form Successful</h2>
```

```
<p>Id:@Model.Id</p>
```

```
<p>Name:@Model.Name</p>
```

```
<p>Salary:@Model.Salary</p>
```

```
<p>Email:@Model.Email</p>
```

```
<p>JoiningDate:@Model.JoiningDate</p>
```

```
*****
```

```
Problem2:
```

```
Product.cs
```

```
using System.ComponentModel.DataAnnotations;
```

```
namespace CollegePortalMvcApp.Models
```

```
{
```

```
    public class Employee
```

```
    {
```

```
        [Required(ErrorMessage = "Id is required")]
```

```
        public required int Id{get;set;}
```

```
        [Required(ErrorMessage = "Enter the required field")]
```

```
        public required string Name{get;set;}
```

```
        [Range(10000,500000, ErrorMessage = "salary should be in between 10k to 500k")]
```

```
        public decimal Salary{get;set;}
```

```

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Invalid email address")]
        public string Email{ get; set; }

```

```

        [Required(ErrorMessage = "Joining date is required")]
        [DataType(DataType.Date)]
        public DateTime JoiningDate{get;set;}
    }
}

```

ProductController.cs

```

using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
using System.Collections.Generic;
using System.Runtime.CompilerServices;

```

```

namespace CollegePortalMvcApp.Controllers
{
    public class ProductsController : Controller
    {
        //Static list in memory storage
        private static List<Products> products = new List<Products>();
        public IActionResult Create()
        {
            return View();
        }
        [HttpPost]
        [HttpPost]
        public IActionResult Create(Products product)
        {
            if (ModelState.IsValid)
            {
                products.Add(product);
                return RedirectToAction("Index");
            }

            return View(product);
        }
        public IActionResult Index()
        {
            return View(products);
        }
    }
}

```

Views -> create.cshtml

@model CollegePortalMvcApp.Models.Products

<h2>Add Product</h2>

<form asp-action="Create" method="post">

<div asp-validation-summary="All"></div>

<div>

<input asp-for="Id" placeholder="Id" />

</div>

```

<div>
    <input asp-for="Name" placeholder="Name" />
    <span asp-validation-for="Name"></span>
</div>

<div>
    <input asp-for="Price" placeholder="Price" />
    <span asp-validation-for="Price"></span>
</div>

<div>
    <input asp-for="Quantity" placeholder="Quantity" />
    <span asp-validation-for="Quantity"></span>
</div>

<button type="submit">Submit</button>

</form>
Index.cshtml

```

```
@model List<CollegePortalMvcApp.Models.Products>
```

```

<h2> table list </h2>
<table border="1">
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Price</th>
        <th>Quantity</th>
    </tr>

```

```

@foreach (var item in Model)
{
    <tr>
        <td>@item.Id</td>
        <td>@item.Name</td>
        <td>@item.Price</td>
        <td>@item.Quantity</td>
    </tr>
}

```

```

</table>
*****

```

Problem3:

Student.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace CollegePortalMvcApp.Models
```

```

{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }

        [Range(18, 60)]

```

```

        public int Age { get; set; }

        [EmailAddress]
        public string Email { get; set; }
    }
}
StudentController.cs
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers
{
    public class StudentController : Controller
    {
        // static list (no database)
        static List<Student> students = new List<Student>()
        {
            new Student { Id = 1, Name = "Ram", Age = 20, Email = "ram@gmail.com" },
            new Student { Id = 2, Name = "Sam", Age = 22, Email = "sam@gmail.com" }
        };

        // GET: Create
        public IActionResult Create()
        {
            return View();
        }

        // POST: Create
        [HttpPost]
        public IActionResult Create(Student student)
        {
            if (ModelState.IsValid)
            {
                student.Id = students.Count + 1;
                students.Add(student);

                return RedirectToAction("Index");
            }

            return View(student);
        }

        // show all students
        public IActionResult Index()
        {
            return View(students);
        }

        // GET: Edit
        public IActionResult Edit(int id)
        {
            var student = students.FirstOrDefault(s => s.Id == id);
            return View(student);
        }

        // POST: Edit
        [HttpPost]
        public IActionResult Edit(Student student)
        {
            var existing = students.FirstOrDefault(s => s.Id == student.Id);

```

```

        if (existing != null)
        {
            existing.Name = student.Name;
            existing.Age = student.Age;
            existing.Email = student.Email;
        }

        return RedirectToAction("Index");
    }
}
}
Views
Create.cshtml
@model CollegePortalMvcApp.Models.Student

```

```
<h2>Create Student</h2>
```

```
<form asp-action="Create" method="post">
```

```

    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>
    <br />

```

```

    <div>
        <label>Age</label>
        <input asp-for="Age" />
    </div>
    <br />

```

```

    <div>
        <label>Email</label>
        <input asp-for="Email" />
    </div>
    <br />

```

```
<button type="submit">Create</button>
```

```
</form>
```

```

Index.cshtml
@model List<CollegePortalMvcApp.Models.Student>

```

```
<h2>Student List</h2>
```

```

<table border="1">
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Age</th>
        <th>Email</th>
        <th>Action</th>
    </tr>

```

```

    @foreach (var item in Model)
    {
        <tr>

```

```

        <td>@item.Id</td>
        <td>@item.Name</td>
        <td>@item.Age</td>
        <td>@item.Email</td>
        <td>
            <a href="/Student/Edit/@item.Id">Edit</a>
        </td>
    </tr>
}
</table>
Edit.cshtml
@model CollegePortalMvcApp.Models.Student

```

```
<h2>Edit Student</h2>
```

```
<form asp-action="Edit" method="post">
```

```

    <!-- hidden id -->
    <input type="hidden" asp-for="Id" />

```

```

<div>
    <label>Name</label>
    <input asp-for="Name" />
</div>
<br />

```

```

<div>
    <label>Age</label>
    <input asp-for="Age" />
</div>
<br />

```

```

<div>
    <label>Email</label>
    <input asp-for="Email" />
</div>
<br />

```

```
<button type="submit">Update</button>
```

```

</form>
*****

```

Problem 4:

Order.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace CollegePortalMvcApp.Models
```

```

{
    public class Order
    {
        public int OrderId { get; set; }

        public string CustomerName { get; set; }

        public double Amount { get; set; }

        public bool IsPremiumCustomer { get; set; }
    }
}

```

```

}
OrderController.cs
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers
{
    public class OrderController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Order order)
        {
            order.OrderId = new Random().Next(1, 1000); // simple id

            return View("Details", order);
        }
    }
}

```

Order views

Create.cshtml

@model CollegePortalMvcApp.Models.Order

<h2>Create Order</h2>

<form asp-action="Create" method="post">

```

<div>
    <label>Customer Name</label>
    <input asp-for="CustomerName" />
</div>
<br />

```

```

<div>
    <label>Amount</label>
    <input asp-for="Amount" />
</div>
<br />

```

```

<div>
    <label>Premium Customer</label>
    <input asp-for="IsPremiumCustomer" type="checkbox" />
</div>
<br />

```

<button type="submit">Submit</button>

</form>

Details.cshtml

@model CollegePortalMvcApp.Models.Order

<h2>Order Details</h2>

<p>Order Id: @Model.OrderId</p>


```
<p>Customer Name: @Model.CustomerName</p>
<p>Amount: @Model.Amount</p>
```

```
@if (Model.IsPremiumCustomer)
{
    <p>Premium Discount Applied 🎉</p>
}
```

```
@if (Model.Amount > 5000)
{
    <p>Eligible for Free Shipping</p>
}
```

Problem:

Employee.cs

namespace CollegePortalMvcApp.Models

```
{
    public class Employee
    {
        public string Name { get; set; }
        public int Age { get; set; }
    }
}
```

EmployeeController.cs

using Microsoft.AspNetCore.Mvc;

using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers

```
{
    public class EmployeeController : Controller
    {
        // form
        public IActionResult Create()
        {
            var list = new List<Employee>()
            {
                new Employee(),
                new Employee(),
                new Employee()
            };

            return View(list);
        }

        // submit
        [HttpPost]
        public IActionResult Create(List<Employee> employees)
        {
            return View("Details", employees);
        }
    }
}
```

Views

Create.cshtml

@model List<CollegePortalMvcApp.Models.Employee>

<h2>Enter Employees</h2>

```

<form asp-action="Create" method="post">

@for (int i = 0; i < Model.Count; i++)
{
    <div>
        <h4>Employee @(i + 1)</h4>

        <label>Name</label>
        <input asp-for="@Model[i].Name" />

        <label>Age</label>
        <input asp-for="@Model[i].Age" />
    </div>
    <br />
}

<button type="submit">Submit</button>

</form>
Details.cshtml
@model List<CollegePortalMvcApp.Models.Employee>

<h2>Employee List</h2>

@foreach (var emp in Model)
{
    <p>Name: @emp.Name</p>
    <p>Age: @emp.Age</p>
    <hr />
}

```

Problem:

User.cs

```

namespace CollegePortalMvcApp.Models
{
    public class User
    {
        public string Username { get; set; }
        public string Password { get; set; }
    }
}

```

UserController.cs

```

using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

```

```

namespace CollegePortalMvcApp.Controllers
{
    public class UserController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(User user)
        {
            // username should not contain "admin"

```

```

        if (user.Username != null && user.Username.ToLower().Contains("admin"))
        {
            ModelState.AddModelError("Username", "username should not contain admin");
        }

        // password length check
        if (user.Password == null || user.Password.Length < 8)
        {
            ModelState.AddModelError("Password", "password must be at least 8 characters");
        }

        // check validation
        if (!ModelState.IsValid)
        {
            return View(user);
        }

        return Content("form submitted successfully");
    }
}
}

```

Views

Create.cshtml

```

using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

```

```

namespace CollegePortalMvcApp.Controllers
{
    public class UserController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(User user)
        {
            // username should not contain "admin"
            if (user.Username != null && user.Username.ToLower().Contains("admin"))
            {
                ModelState.AddModelError("Username", "username should not contain admin");
            }

            // password length check
            if (user.Password == null || user.Password.Length < 8)
            {
                ModelState.AddModelError("Password", "password must be at least 8 characters");
            }

            // check validation
            if (!ModelState.IsValid)
            {
                return View(user);
            }

            return Content("form submitted successfully");
        }
    }
}

```

```

    }
}
}
*****

```

Problem:

Product.cs

```

namespace CollegePortalMvcApp.Models
{
    public class Product
    {
        public string Name { get; set; }
        public int Price { get; set; }
    }
}

```

ProductContoller.cs

```

using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
using System.Collections.Generic;

```

```

namespace CollegePortalMvcApp.Controllers
{
    public class ProductController : Controller
    {
        private static List<Product> products = new List<Product>();

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Product product)
        {
            if (ModelState.IsValid)
            {
                products.Add(product);
                return RedirectToAction("Index");
            }

            return View(product);
        }

        public IActionResult Index()
        {
            return View(products);
        }
    }
}

```

Views

Create.cshtml

```
@model CollegePortalMvcApp.Models.Product
```

```
<h2>Create Product</h2>
```

```
<form asp-action="Create" method="post">
```

```
<div>
```

```
<label>Name</label>
```

```

        <input asp-for="Name" />
    </div>
    <br />

    <div>
        <label>Price</label>
        <input asp-for="Price" />
    </div>
    <br />

    <button type="submit" name="action" value="preview">Preview</button>
    <button type="submit" name="action" value="save">Save</button>

```

```
</form>
```

Index.cshtml

```
@model List<CollegePortalMvcApp.Models.Product>
```

```
<h2>Saved Products</h2>
```

```
@foreach (var p in Model)
```

```
{
    <p>Name: @p.Name</p>
    <p>Price: @p.Price</p>
    <hr />
}
```

```
*****
```

Problem

Product.cs

```
namespace CollegePortalMvcApp.Models
```

```
{
    public class Product
    {
        public int Id { get; set; } // important
        public string Name { get; set; }
        public int Price { get; set; }
    }
}
```

ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;
```

```
namespace CollegePortalMvcApp.Controllers
```

```
{
    public class ProductController : Controller
    {
        static List<Product> products = new List<Product>()
        {
            new Product { Id = 1, Name = "Pen", Price = 10 },
            new Product { Id = 2, Name = "Book", Price = 50 }
        };
    }
}
```

```
// list
```

```
public IActionResult Index()
{
    return View(products);
}
```

```
// edit get
```

```

public IActionResult Edit(int id)
{
    var product = products.FirstOrDefault(p => p.Id == id);
    return View(product);
}

// edit post
[HttpPost]
public IActionResult Edit(Product product)
{
    var existing = products.FirstOrDefault(p => p.Id == product.Id);

    if (existing != null)
    {
        existing.Name = product.Name;
        existing.Price = product.Price;
    }

    return RedirectToAction("Index");
}
}

```

Views

Index.cshtml

@model List<CollegePortalMvcApp.Models.Product>

<h2>Product List</h2>

```

<table border="1">
  <tr>
    <th>Id</th>
    <th>Name</th>
    <th>Price</th>
    <th>Action</th>
  </tr>

```

@foreach (var item in Model)

```

{
  <tr>
    <td>@item.Id</td>
    <td>@item.Name</td>
    <td>@item.Price</td>
    <td>
      <a href="/Product/Edit/@item.Id">Edit</a>
    </td>
  </tr>
}

```

</table>

Edit.cshtml

@model CollegePortalMvcApp.Models.Product

<h2>Edit Product</h2>

<form asp-action="Edit" method="post">

<input type="hidden" asp-for="Id" />

<div>

```

        <label>Name</label>
        <input asp-for="Name" />
    </div>
    <br />

    <div>
        <label>Price</label>
        <input asp-for="Price" />
    </div>
    <br />

    <button type="submit">Update</button>

</form>
*****

Problem
UserForm.cs
namespace CollegePortalMvcApp.Models
{
    public class UserForm
    {
        public string Name { get; set; }
        public string Gender { get; set; }
        public List<string> Skills { get; set; }
    }
}

UserFormController.cs
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers
{
    public class UserFormController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(UserForm model)
        {
            // simple validation
            if (string.IsNullOrEmpty(model.Name))
            {
                ModelState.Remove("Name");
                ModelState.AddModelError("Name", "name is required");
            }

            if (string.IsNullOrEmpty(model.Gender))
            {
                ModelState.AddModelError("Gender", "select gender");
            }

            if (model.Skills == null || model.Skills.Count == 0)
            {
                ModelState.Remove("Skills");
                ModelState.AddModelError("Skills", "select at least one skill");
            }
        }
    }
}

```

```

    }

    if (!ModelState.IsValid)
    {
        return View(model);
    }

    return Content("submitted successfully");
}
}
}
Create.cshtml
@model CollegePortalMvcApp.Models.UserForm

```

<h2>User Form</h2>

<form asp-action="Create" method="post">

```

<div>
    <label>Name</label>
    <input asp-for="Name" />
    <span asp-validation-for="Name"></span>
</div>
<br />

```

```

<div>
    <label>Gender</label>
    <select asp-for="Gender">
        <option value="">--select--</option>
        <option value="Male">Male</option>
        <option value="Female">Female</option>
    </select>
    <span asp-validation-for="Gender"></span>
</div>
<br />

```

```

<div>
    <label>Skills</label><br />

    <input type="checkbox" name="Skills" value="C#"
    @(Model?.Skills != null && Model.Skills.Contains("C#") ? "checked" : "") /> C# <br />

    <input type="checkbox" name="Skills" value="Java"
    @(Model?.Skills != null && Model.Skills.Contains("Java") ? "checked" : "") /> Java <br />

    <input type="checkbox" name="Skills" value="Python"
    @(Model?.Skills != null && Model.Skills.Contains("Python") ? "checked" : "") /> Python <br />

    @Html.ValidationMessage("Skills")
</div>
<br />

```

<button type="submit">Submit</button>

</form>

Scenario Based Questions

1. because model binding needs matching names for nested objects, but form is using City and State instead of Address.City and Address.State, so Address is not bound and becomes null.
2. the clicked button sends its name=value pair with the form, so MVC receives action=save or action=delete and uses that to decide which logic to run.
3. model binding fails to convert "twenty" into an int, so MVC adds a model error automatically and ModelState becomes invalid.
4. returned just View() and didn't send the model, so data is lost and it shows null instead of modified value
5. when checkbox is unchecked it doesn't send any value in request, so model binding doesn't get IsSubscribed and sets default value = false automatically
6. form doesn't send Quantity, so model binding sets it to default int value (0) and overwrites the initial value 10
7. Route Id will be 5 and Model Id will be 10 because id comes from URL route while p.Id comes from form data, so both are bound separately causing mismatch
8. because ViewBag is temporary and only works for the same request, but RedirectToAction creates a new request so data is lost and becomes null

Library Management System

Book.cs

using System.ComponentModel.DataAnnotations;

namespace CollegePortalMvcApp.Models

```
{
    public class Book
    {
        public int Id { get; set; }

        [Required]
        public required string Title { get; set; }

        [Required]
        public required string Author { get; set; }

        [Range(100, 5000)]
        public int Price { get; set; }

        public int CategoryId { get; set; }

        public Category? Category { get; set; }
    }
}
```

Category.cs

using System.ComponentModel.DataAnnotations;

namespace CollegePortalMvcApp.Models

```
{
    public class Category
    {
        public int Id { get; set; }

        [Required]
        public required string Name { get; set; }

        public ICollection<Book>? Books { get; set; }
    }
}
```

Course.cs

using System.ComponentModel.DataAnnotations;

namespace CollegePortalMvcApp.Models

```
{
```

```

public class Course
{
    public int Id { get; set; }

    [Required]
    public required string Title { get; set; }

    public ICollection<Student>? Students { get; set; }
}
}
//updated Db context
using Microsoft.EntityFrameworkCore;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Book> Books { get; set; }
        public DbSet<Category> Categories { get; set; }
        public DbSet<Course> Courses { get; set; }
    }
}

```

```

BookController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using CollegePortalMvcApp.Data;
using CollegePortalMvcApp.Models;

namespace StudentMvcApp.Controllers
{
    public class BookController : Controller
    {
        private readonly AppDbContext _context;

        public BookController(AppDbContext context)
        {
            _context = context;
        }

        // Read + LINQ + Eager Loading
        public async Task<ActionResult> Index()
        {
            var books = await _context.Books
                .Include(b => b.Category)
                .OrderByDescending(b => b.Price)
                .ToListAsync();

            ViewBag.TotalBooks = books.Count;
            ViewBag.MaxPrice = books.Max(b => b.Price);

            return View(books);
        }
    }
}

```

```

}

// Get Create
public IActionResult Create()
{
    ViewBag.Categories = _context.Categories.ToList();
    return View();
}

// Post Create
[HttpPost]
public async Task<IActionResult> Create(Book book)
{
    if (ModelState.IsValid)
    {
        _context.Add(book);
        await _context.SaveChangesAsync();

        TempData["Message"] = "Book Added!";
        HttpContext.Session.SetString("LastBook", book.Title);

        return RedirectToAction("Index");
    }

    ViewBag.Categories = _context.Categories.ToList();
    return View(book);
}

// Get
public async Task<IActionResult> Edit(int id)
{
    var book = await _context.Books.FindAsync(id);
    ViewBag.Categories = _context.Categories.ToList();
    return View(book);
}

// Post Edit
[HttpPost]
public async Task<IActionResult> Edit(Book book)
{
    if (ModelState.IsValid)
    {
        _context.Update(book);
        await _context.SaveChangesAsync();

        TempData["Message"] = "Book Updated!";
        return RedirectToAction("Index");
    }

    return View(book);
}

// Delete
public async Task<IActionResult> Delete(int id)
{
    var book = await _context.Books.FindAsync(id);
    if (book != null)
    {

```

```

        _context.Remove(book);
        await _context.SaveChangesAsync();
    }

    return RedirectToAction("Index");
}

// Custom route
public IActionResult ByCategory(int id)
{
    var books = _context.Books
        .Include(b => b.Category)
        .Where(b => b.CategoryId == id)
        .ToList();

    return View("Index", books);
}
}
}

// Seed Data
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    context.Database.Migrate();
    if (!context.Categories.Any())
    {
        context.Categories.AddRange(
            new Category { Name = "Programming" },
            new Category { Name = "Science" },
            new Category { Name = "Mathematics" }
        );
        context.SaveChanges();
    }
}

```

CourseController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using CollegePortalMvcApp.Models;
using CollegePortalMvcApp.Data;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers
{
    public class CourseController : Controller
    {
        private readonly AppDbContext _context;

        public CourseController(AppDbContext context)
        {
            _context = context;
        }

        // Index
        public async Task<IActionResult> Index()
        {
            var courses = await _context.Courses.ToListAsync();
            return View(courses);
        }
    }
}

```

```

    }

    // Details
    public async Task<IActionResult> Details(int id)
    {
        var course = await _context.Courses
            .Include(c => c.Students)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (course == null) return NotFound();

        return View(course);
    }

    // Create GET
    public IActionResult Create()
    {
        return View();
    }

    // Create Post
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(Course course)
    {
        if (ModelState.IsValid)
        {
            _context.Courses.Add(course);
            await _context.SaveChangesAsync();

            TempData["Success"] = "Course added successfully!";
            return RedirectToAction(nameof(Index));
        }

        return View(course);
    }

    // Delete
    public async Task<IActionResult> Delete(int id)
    {
        var course = await _context.Courses.FindAsync(id);
        if (course == null) return NotFound();

        _context.Courses.Remove(course);
        await _context.SaveChangesAsync();

        TempData["Success"] = "Course deleted successfully!";
        return RedirectToAction(nameof(Index));
    }
}
}
}

//Custom Routing in program.cs
//Custom Routing
app.MapControllerRoute(
    name: "booksByCategory",
    pattern: "Books/Category/{id}",
    defaults: new { controller = "Book", action = "ByCategory" });
Views

```

Create.cshtml

@model CollegePortalMvcApp.Models.Book

<h2>Create Book</h2>

<form asp-action="Create" method="post">

<div>

<label>Title</label>

<input asp-for="Title" />

</div>

<div>

<label>Author</label>

<input asp-for="Author" />

</div>

<div>

<label>Price</label>

<input asp-for="Price" />

</div>

<div>

<label>Category</label>

<select asp-for="CategoryId">

@foreach (var cat in ViewBag.Categories)

{

<option value="@cat.Id">@cat.Name</option>

}

</select>

</div>

<button type="submit">Save</button>

</form>

Edit.cshtml

@model CollegePortalMvcApp.Models.Book

<h2>Edit Book</h2>

<form asp-action="Edit" method="post">

<input type="hidden" asp-for="Id" />

<div>

<label>Title</label>

<input asp-for="Title" />

</div>

<div>

<label>Author</label>

<input asp-for="Author" />

</div>

<div>

<label>Price</label>

<input asp-for="Price" />

</div>

<div>

<label>Category</label>

```

        <select asp-for="CategoryId">
            @foreach (var cat in ViewBag.Categories)
            {
                <option value="@cat.Id">@cat.Name</option>
            }
        </select>
    </div>

    <button type="submit">Update</button>
</form>
Index.cshtml
@model IEnumerable<CollegePortalMvcApp.Models.Book>

<h2>Book List</h2>

@if (TempData["Message"] != null)
{
    <p style="color:green">@TempData["Message"]</p>
}

<p>Total Books: @ViewBag.TotalBooks</p>
<p>Max Price: @ViewBag.MaxPrice</p>

@if (Context.Session.GetString("LastBook") != null)
{
    <p>Last Added Book: @Context.Session.GetString("LastBook")</p>
}

<a asp-action="Create">Add Book</a>

<table border="1">
    <tr>
        <th>Title</th>
        <th>Author</th>
        <th>Price</th>
        <th>Category</th>
        <th>Action</th>
    </tr>

    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Title</td>
            <td>@item.Author</td>
            <td>@item.Price</td>
            <td>@item.Category?.Name</td>
            <td>
                <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
                <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
            </td>
        </tr>
    }
</table>

```

Online Store (Orders + Products)

Models

Product.cs

using System.ComponentModel.DataAnnotations;

```
namespace CollegeWindowMvcApp.Models
```

```
{  
    public class Product  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public required string Name { get; set; }  
  
        public int Price { get; set; }  
  
        public int StockQuantity { get; set; }  
    }  
}
```

```
Order.cs
```

```
namespace CollegeWindowMvcApp.Models
```

```
{  
    public class Order  
    {  
        public int Id { get; set; }  
  
        public DateTime OrderDate { get; set; }  
  
        public int TotalAmount { get; set; }  
  
        public int ProductId { get; set; }  
  
        public Product? Product { get; set; }  
    }  
}
```

```
Data(AppDbContext.cs)
```

```
using Microsoft.EntityFrameworkCore;  
using CollegeWindowMvcApp.Models;
```

```
namespace CollegeWindowMvcApp.Data
```

```
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options)  
            : base(options)  
        {  
        }  
  
        public DbSet<Product> Products { get; set; }  
        public DbSet<Order> Orders { get; set; }  
    }  
}
```

```
Program.cs
```

```
using Microsoft.EntityFrameworkCore;  
using CollegeWindowMvcApp.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.  
builder.Services.AddControllersWithViews();
```

```
// ADD THIS (Database connection)
```



```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite("Data Source=store.db"));
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
```

```
if (!app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseExceptionHandler("/Home/Error");
```

```
    app.UseHsts();
```

```
}
```

```
app.UseHttpsRedirection();
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

```
// start directly with Product
```

```
app.MapControllerRoute(
```

```
    name: "default",
```

```
    pattern: "{controller=Product}/{action=Index}/{id?}");
```

```
app.Run();
```

```
Controllers
```

```
ProductController.cs
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using CollegeWindowMvcApp.Data;
```

```
using CollegeWindowMvcApp.Models;
```

```
namespace OnlineStoreApp.Controllers
```

```
{
```

```
    public class ProductController : Controller
```

```
    {
```

```
        private readonly AppDbContext _context;
```

```
        public ProductController(AppDbContext context)
```

```
        {
```

```
            _context = context;
```

```
        }
```

```
        public IActionResult Index()
```

```
        {
```

```
            return View(_context.Products.ToList());
```

```
        }
```

```
        public IActionResult Create()
```

```
        {
```

```
            return View();
```

```
        }
```

```
        [HttpPost]
```

```
        public IActionResult Create(Product product)
```

```
        {
```

```
            if (ModelState.IsValid)
```

```
            {
```

```
                _context.Add(product);
```

```

        _context.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(product);
}
}
}
OrderController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using CollegeWindowMvcApp.Data;
using CollegeWindowMvcApp.Models;

namespace CollegeWindowMvcApp.Controllers
{
    public class OrderController : Controller
    {
        private readonly ApplicationDbContext _context;

        public OrderController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var orders = _context.Orders
                .Include(o => o.Product)
                .ToList();

            ViewBag.TotalRevenue = orders.Sum(o => o.TotalAmount);

            ViewBag.LowStock = _context.Products
                .Where(p => p.StockQuantity < 5)
                .ToList();

            ViewBag.Grouped = _context.Orders
                .Include(o => o.Product)
                .AsEnumerable()
                .GroupBy(o => o.Product.Name)
                .Select(g => new
                {
                    Product = g.Key,
                    Count = g.Count()
                })
                .ToList();

            return View(orders);
        }

        public IActionResult Create()
        {
            ViewBag.Products = _context.Products.ToList();
            return View();
        }
    }
}

```

```

[HttpPost]
public IActionResult Create(Order order)
{
    var product = _context.Products.Find(order.ProductId);

    if (product != null && product.StockQuantity > 0)
    {
        order.OrderDate = DateTime.Now;
        order.TotalAmount = product.Price;

        product.StockQuantity--;

        _context.Add(order);
        _context.SaveChanges();

        return RedirectToAction("Index");
    }

    ViewBag.Products = _context.Products.ToList();
    return View(order);
}
}
}
Views
Product
Create.html
@model CollegeWindowMvcApp.Models.Product

<h2>Create Product</h2>

<form asp-action="Create" method="post">
    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>

    <div>
        <label>Price</label>
        <input asp-for="Price" />
    </div>

    <div>
        <label>Stock</label>
        <input asp-for="StockQuantity" />
    </div>

    <button type="submit">Save</button>
</form>
Index.html
@model IEnumerable<CollegeWindowMvcApp.Models.Product>

<h2>Product List</h2>

<a asp-action="Create">Add Product</a>

<table border="1">
    <tr>
        <th>Name</th>

```

```

        <th>Price</th>
        <th>Stock</th>
    </tr>

    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Name</td>
            <td>@item.Price</td>
            <td>@item.StockQuantity</td>
        </tr>
    }
</table>
Order
Create.cshtml
@model CollegeWindowMvcApp.Models.Order

<h2>Create Order</h2>

<form asp-action="Create" method="post">

    <div>
        <label>Product</label>
        <select asp-for="ProductId">
            @foreach (var p in ViewBag.Products)
            {
                <option value="@p.Id">@p.Name</option>
            }
        </select>
    </div>

    <button type="submit">Place Order</button>

</form>
Index.cshtml
@model IEnumerable<CollegeWindowMvcApp.Models.Order>

<h2>Orders</h2>

@if (TempData["Message"] != null)
{
    <p style="color:green">@TempData["Message"]</p>
}

<p>Total Revenue: @ViewBag.TotalRevenue</p>

<a asp-action="Create">Place Order</a>

<table border="1">
    <tr>
        <th>Product</th>
        <th>Total</th>
        <th>Date</th>
        <th>Action</th>
    </tr>

    @foreach (var item in Model)
    {

```

```

        <tr>
            <td>@item.Product?.Name</td>
            <td>@item.TotalAmount</td>
            <td>@item.OrderDate</td>
            <td>
                <a asp-action="Details" asp-route-id="@item.Id">Details</a>
            </td>
        </tr>
    }
</table>

```

```

@if (ViewBag.LowStock != null)
{
    <h3>Low Stock Products</h3>
    <ul>
        @foreach (var p in ViewBag.LowStock)
        {
            <li>@p.Name (@p.StockQuantity)</li>
        }
    </ul>
}

```

```

@if (ViewBag.Grouped != null)
{
    <h3>Orders Grouped</h3>
    <ul>
        @foreach (var g in ViewBag.Grouped)
        {
            <li>@g.Product - @g.Count</li>
        }
    </ul>
}

```

Details.cshtml

```
@model CollegeWindowMvcApp.Models.Order
```

```
<h2>Order Details</h2>
```

```

<p>Product: @Model.Product?.Name</p>
<p>Total: @Model.TotalAmount</p>
<p>Date: @Model.OrderDate</p>

```